Pipes in the Desert
Detailed Plan

E5 - TEAM A

Supervisor:

Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

April 18th, 2026

8. Detailed plans

8.0 Corrections

8.0.1 Input/Output mode selection (7.1.1 and 7.1.2)

Issue: The previous document did not describe how the user selects between file-based input and console input, and did not specify where output files are saved.

Correction: The following two commands are added to Section 8.1.2 Input Language:

load <filepath>
Description: Loads commands from the specified input file line by line instead of reading from the keyboard. All subsequent input is read from the file until EOF.

save <filepath>
Description: Redirects all subsequent output to the specified file. Output files are saved to the current working directory from which the program is launched, using the filename provided. If no save command is issued, all output is printed to the console only.

Result: The input/output mode switching mechanism is now explicitly defined in the input language and the output file location is specified.

8.0.2 Wrong input handling in use case scenarios (7.2)

Issue: Cases of wrong input were not included in the use case scenarios.

Correction: A wrong input clause has been added to the dialog/scenario of all relevant use cases in Section 8.2. In any use case where the Tester selects a numbered menu option or enters an ID, the following rule applies: if the Tester enters a value outside the valid options, a non-numeric string, or an out-of-range number, the system prints the corresponding error message from Section 7.1.3.12 and re-prompts the same input without consuming the turn.

This rule applies to all menu selections, ID inputs, and command inputs in the use cases.

Result: All use case scenarios now explicitly cover the wrong input path, making them complete and testable.

8.1 Design level plan of classes

8.1.1 System

- **Responsibility**
  Manages the overall game flow. Initializes the game, controls turns, updates timer, triggers events, and ends the game.

- **Superclasses**
  None

● **Interfaces**
None

● **Attributes**
○ teams: List<Team> - Stores both teams. (private)
○ network: PipeNetwork - Represents the game network. (private)
○ timer: GameTimer - Controls game duration. (private)
○ scoreBoard: ScoreBoard - Tracks scores. (private)
○ currentPlayer: Player - Active player. (private)
○ running: boolean - Game state flag. (private, initial value = false)
○ waterFlowManager: WaterFlowManager - Handles water flow. (private)
○ turnOrder: List<Player> - Player sequence. (private, initial value = empty list)
○ currentTurnIndex: int - Tracks turn position. (private, initial value = -1)

● **Methods**
○ void startGame()
Sets running = true.
Calls initializeGame().
If timer is not null, calls timer.start().
If turnOrder is not empty, sets currentTurnIndex = 0, sets currentPlayer = turnOrder.get(0), calls currentPlayer.startTurn().

○ void initializeGame()
Creates Spring(id:1), Cistern(id:2), Pump(id:3), Pipe(id:4), Pipe(id:5) and adds all five to the network.
Connects: Spring↔Pipe4, Pipe4↔Pump3, Pump3↔Pipe5, Pipe5↔Cistern2.
Sets Spring's output pipe to Pipe4.
Registers Pipe4 and Pipe5 on the pump via addConnectedPipe().
Sets pump direction to Pipe4→Pipe5.
Calls rebuildTurnOrder().
Places every player on Pump3 via setPosition().

○ void nextTurn()
Calls currentPlayer.endTurn().
Increments currentTurnIndex by 1 modulo turnOrder.size().
Sets currentPlayer = turnOrder.get(currentTurnIndex).
Calls currentPlayer.startTurn().
If currentTurnIndex wraps back to 0 and timer is active, calls timer.tick().
Note: the timer decrements once per complete round of all players, not once per individual turn.

○ void updateRoundEvents()
Calls requestComponentGeneration().
Retrieves working pumps from the network.
If the list is not empty, prompts "Trigger random pump breakdown? (yes/no):".
If the answer is yes, calls breakDown() on the first working pump in the list.
Calls waterFlowManager.recalculateFlow() once, accounting for all state changes from this tick combined.

○ void requestComponentGeneration()
If network is null, returns immediately.

Iterates over all cisterns in the network.
For each cistern: prompts "Generate new pipe at cistern? (yes/no):". If yes, calls cistern.generatePipe().
Then prompts "Generate new pump at cistern? (yes/no):". If yes, calls cistern.generatePump().

○ void endGame()
Sets running = false.
Calls timer.stop().
Calls scoreBoard.determineWinner().
If the result is null, prints "Game ended in a draw."
Otherwise prints "Winner: " followed by the winning team's name.

○ boolean checkEndCondition() - Returns true if timer is not null and timer.hasExpired() is true. Otherwise returns false.
○ boolean isRunning() - Returns running.
○ Player getCurrentPlayer() - Returns currentPlayer.
○ PipeNetwork getNetwork() - Returns network.
○ ScoreBoard getScoreBoard() - Returns scoreBoard.
○ GameTimer getTimer() - Returns timer.
○ WaterFlowManager getWaterFlowManager() - Returns waterFlowManager.

8.1.2 Player

● **Responsibility**
Represents a player in the game. Handles movement, turn control, and stores player state.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**
○ name: String - Player name. (protected)
○ position: NetworkElement - Current position in network. (protected)
○ team: Team - Player’s team. (protected)
○ hasActedThisTurn: boolean - Tracks if player has acted. (protected, initial value = false)

● **Methods**

○ boolean moveTo(NetworkElement target)
If canAct() returns false, return false.
If target is null, return false.
If target is a Pipe:
If the pipe is already occupied via isOccupied(), return false.
If the player's current position is a Pipe, call clearOccupant() on it.
Set position = target. Call target.setOccupant(this). Return true.
If target is a Pump:
If the player's current position is a Pipe, call clearOccupant() on it.

Set position = target. Return true.
If target is any other type (Spring, Cistern), return false.
Note: adjacency is not enforced - any valid pipe or pump ID in the network can be selected.

○ void startTurn() - Sets hasActedThisTurn = false.
○ void endTurn() - Sets hasActedThisTurn = true.
○ boolean canAct() - Returns !hasActedThisTurn.

○ void setPosition(NetworkElement e)
If the current position is a Pipe, calls clearOccupant() on it.
Sets position = e.
If the new position is a Pipe, calls setOccupant(this) on it.

○ NetworkElement getPosition() - Returns position.
○ String getName() - Returns name.
○ Team getTeam() - Returns team.

8.1.3 Plumber

● **Responsibility**
Represents a plumber player. Performs actions such as repairing pipes and pumps, connecting pipe ends, and placing or inserting new components.

● **Superclasses**
Player

● **Interfaces**
None

● **Attributes**
○ carriedPump: boolean - Indicates if plumber is carrying a pump. (private, initial value = false)
○ carriedPipe: boolean - Indicates if plumber is carrying a pipe. (private, initial value = false)

● **Methods**
○ void repairPipe(Pipe pipe) - Repairs a punctured pipe.
○ void repairPump(Pump pump) - Repairs a broken pump.
○ void changePumpDirection(Pump pump, Pipe input, Pipe output) - Changes the direction of water flow in a pump.
○ void connectPipeEnd(Pipe pipe, NetworkElement target) - Connects a free end of a pipe to another element.
○ void collectPumpFromCistern(Cistern c) - Collects a pump from a cistern if available.
○ void collectPipeFromCistern(Cistern c) - Collects a pipe from a cistern if available.
○ void insertPump(Pipe pipe, Pump pump, PipeNetwork network) - Inserts a pump into an existing pipe.
○ void placeNewPipe(Pipe pipe, NetworkElement a, NetworkElement b, PipeNetwork network) - Places and connects a new pipe in the network.

8.1.4 Saboteur

● **Responsibility**
Represents a saboteur player. Performs actions that disrupt the network by causing leaks and altering water flow.

● **Superclasses**
Player

● **Interfaces**
None

● **Attributes**
None

● **Methods**
○ void puncturePipe(Pipe pipe) - Punctures a pipe causing water leakage.
○ void changePumpDirection(Pump pump, Pipe input, Pipe output) - Changes pump direction to disrupt water flow.

8.1.5 Pipe

● Responsibility
Represents a pipe in the network that carries water. It can be punctured, have a free end, and can be occupied by a player.

● **Superclasses**
NetworkElement

● **Interfaces**
None

● **Attributes**
○ punctured: boolean - Indicates if the pipe is leaking. (private, initial value = false)
○ freeEnd: boolean - Indicates if one end is disconnected. (private, initial value = false)
○ currentWater: int - Amount of water currently in the pipe. (private, initial value = 0)
○ capacity: int - Maximum water capacity. (private)
○ occupiedBy: Player - Player currently on the pipe. (private, initial value = null)

● **Methods**
○ void puncture() - Marks pipe as punctured.
○ void repair() - Repairs the pipe.
○ boolean isPunctured() - Returns puncture state.
○ boolean hasFreeEnd() - Returns connection state.
○ void connectFreeEnd(NetworkElement target) - Connects pipe to another element.
○ boolean isOccupied() - Checks if pipe is occupied.
○ Player getOccupant() - Returns occupying player.
○ void disconnectEnd() - Disconnects one end of the pipe.
○ void setOccupant(Player player) - Assigns player to pipe.
○ void clearOccupant() - Removes player from pipe.
○ int getCapacity() - Returns pipe capacity.
○ int getCurrentWater() - Returns current water.

○ void setCurrentWater(int value) - Updates water amount.

8.1.6 Pump

● **Responsibility**
Represents a pump that routes water between pipes. It can change direction and may become broken.

● **Superclasses**
NetworkElement

● **Interfaces**
None

● **Attributes**
○ connectedPipes: List<Pipe> - Pipes connected to the pump. (private, initial value = empty list)
○ activeInput: Pipe - Current input pipe. (private, initial value = null)
○ activeOutput: Pipe - Current output pipe. (private, initial value = null)
○ broken: boolean - Indicates if pump is broken. (private, initial value = false)
○ buffer: int - Temporary water storage. (private, initial value = 0)
○ connectionLimit: int - Maximum allowed connections. (private)

● **Methods**
○ void setDirection(Pipe input, Pipe output) - Sets input and output pipes.
○ void breakDown() - Marks pump as broken.
○ void repair() - Repairs the pump.
○ boolean isBroken() - Returns pump state.
○ void addConnectedPipe(Pipe pipe) - Adds a pipe connection.
○ boolean canTransferWater() - Checks if water can flow through pump.
○ Pipe getActiveOutput() - Returns output pipe.
○ boolean canAcceptConnection() - Checks if new pipe can be connected.
○ void setBuffer(int value) - Updates internal water buffer.
○ int getBuffer() - Returns buffer value.
○ int getConnectionLimit() - Returns max connections.
○ List<Pipe> getConnectedPipes() - Returns connected pipes.
○ Pipe getActiveInput() - Returns input pipe.

8.1.7 PipeNetwork

● **Responsibility**
Maintains all network elements and their connections. Handles adding, removing, and connecting elements.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**

○ elements: List<NetworkElement> - Stores all elements. (private, initial value = empty list)
○ pipes: List<Pipe> - Stores all pipes. (private, initial value = empty list)
○ pumps: List<Pump> - Stores all pumps. (private, initial value = empty list)
○ springs: List<Spring> - Stores all springs. (private, initial value = empty list)
○ cisterns: List<Cistern> - Stores all cisterns. (private, initial value = empty list)

● **Methods**
○ void insertPump(Pipe targetPipe, Pump pump)
Capture the original neighbor list of targetPipe before any changes.
If a player is standing on targetPipe, call occupant.setPosition(pump) and clearOccupant() on the pipe.
Call removeElement(targetPipe) to remove it from the network.
Disconnect targetPipe from all original neighbors by calling removeNeighbor() on both sides.
Create two new pipe segments:
segment1 with id = targetPipe.id * 100 + 1
segment2 with id = targetPipe.id * 100 + 2
Both use the same capacity as the original pipe.
Call addElement(pump), addElement(segment1), addElement(segment2).
Call connectElements(segment1, pump) and connectElements(pump, segment2).
Reconnect original neighbors: originalNeighbors.get(0) to segment1, originalNeighbors.get(1) to segment2.
Call pump.addConnectedPipe(segment1) and pump.addConnectedPipe(segment2).
Call pump.setDirection(segment1, segment2).
If any Spring in the network had targetPipe as its output pipe, update that Spring's output pipe to segment1.

8.1.8 WaterFlowManager

● **Responsibility**
Handles water flow simulation in the network. Determines whether water is delivered to cisterns or leaked, and updates the scoreboard accordingly.
cisterns or leaked, and updates the scoreboard accordingly.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**
○ network: PipeNetwork - Represents the current network. (private)
○ scoreBoard: ScoreBoard - Stores and updates scores. (private)

● **Methods**

○ void recalculateFlow()

Retrieves all springs from the network.
Calls processSpring(spring) for each spring in the list.

○ void processSpring(Spring spring)
Get water = spring.produceWater().
Get currentPipe = spring.getOutputPipe().
If currentPipe is null, call registerLeakedWater(water) and return.
Loop while currentPipe is not null:
If currentPipe.isPunctured() or currentPipe.hasFreeEnd() is true:
Call registerLeakedWater(water) and return.
Iterate the neighbors of currentPipe:
If a neighbor is a Cistern:
Call cistern.receiveWater(water) and registerDeliveredWater(water) and return.
If a neighbor is a Pump, store it as nextElement.
If nextElement is a Pump:
If pump.isBroken() or pump.canTransferWater() returns false:
Call registerLeakedWater(water) and return.
Set currentPipe = pump.getActiveOutput() and continue the loop.
If no Cistern or Pump was found among neighbors:
Call registerLeakedWater(water) and return.
If the loop exits without returning, call registerLeakedWater(water).

○ void registerDeliveredWater(int amount) - Calls scoreBoard.addDeliveredWater(amount).
○ void registerLeakedWater(int amount) - Calls scoreBoard.addLeakedWater(amount).
○ PipeNetwork getNetwork() - Returns network.
○ ScoreBoard getScoreBoard() - Returns scoreBoard.
○ void setNetwork(PipeNetwork network) - Sets network.
○ void setScoreBoard(ScoreBoard scoreBoard) - Sets scoreBoard.

8.1.9 ScoreBoard

● **Responsibility**
Stores and manages scores of both teams and determines the winner.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**

○ plumberScore: int - Amount of water delivered to cisterns. (private, initial value = 0)
○ saboteurScore: int - Amount of leaked water. (private, initial value = 0)
○ plumberTeam: Team - Reference to plumber team. (private)
○ saboteurTeam: Team - Reference to saboteur team. (private)

● **Methods**

○ void addDeliveredWater(int amount) - Increases plumber score.
○ void addLeakedWater(int amount) - Increases saboteur score.
○ int getPlumberScore() - Returns plumber score.
○ int getSaboteurScore() - Returns saboteur score.

○ Team determineWinner() - Returns the winning team or null if draw.

8.1.10 GameTimer

● **Responsibility**
Manages the game time and determines when the game ends.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**
○ remainingTime: int - Time left in the game. (private)
○ maxTime: int - Initial game duration. (private)
○ active: boolean - Indicates if the timer is running. (private, initial value = false)

● **Methods**
○ void start() - Starts the timer.
○ void tick() - Decreases remaining time.
○ void stop() - Stops the timer.
○ boolean hasExpired() - Returns true if time is over.
○ int getRemainingTime() - Returns remaining time.
○ void reset(int maxTime) - Resets the timer with a new duration.
○ boolean isActive() - Returns if the timer is running.
○ int getMaxTime() - Returns maximum duration.

8.1.11 Cistern

● Responsibility
Represents a destination node that receives and stores water. It also generates pipes and pumps for players.

● **Superclasses**
NetworkElement

● **Interfaces**
None

● **Attributes**

○ storedWater: int - Total water stored in the cistern. (private, initial value = 0)
○ availablePipes: int - Number of pipes available for pickup. (private, initial value = 0)
○ availablePumps: int - Number of pumps available for pickup. (private, initial value = 0)

● **Methods**

○ void receiveWater(int amount) - Adds water to storedWater.
○ void generatePipe() - Increases available pipe count.
○ void generatePump() - Increases available pump count.

○ boolean hasAvailablePipe() - Checks if pipes are available.
○ boolean hasAvailablePump() - Checks if pumps are available.
○ int getAvailablePipes() - Returns available pipes.
○ int getAvailablePumps() - Returns available pumps.
○ boolean takePipe() - Removes one pipe if available and returns success.
○ boolean takePump() - Removes one pump if available and returns success.

8.1.12 Spring

● **Responsibility**
Represents a water source that generates water and sends it into the network.

● **Superclasses**
NetworkElement

● **Interfaces**
None

● **Attributes**

○ waterOutput: int - Amount of water produced each cycle. (private, initial value = 10)
○ outputPipe: Pipe - Pipe through which water enters the network. (private, initial value = null)

● **Methods**

○ int produceWater() - Returns generated water amount.
○ int getWaterOutput() - Returns current output value.
○ void setWaterOutput(int value) - Updates water output.
○ void setOutputPipe(Pipe pipe) - Assigns output pipe.
○ Pipe getOutputPipe() - Returns output pipe.

8.1.13 Team

● **Responsibility**
Represents a group of players belonging to the same side (Plumbers or Saboteurs). Manages team members.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**

○ name: String - Name of the team. (private)
○ members: List<Player> - Players in the team. (private, initial value = empty list)

● **Methods**

○ void addMember(Player player) - Adds a player to the team.
○ List<Player> getMembers() - Returns all players.
○ String getName() - Returns team name.
○ int getSize() - Returns number of players.

8.1.14 NetworkElement

● **Responsibility**
Abstract base class for all network elements. Manages connections between elements.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**

○ id: int - Unique identifier of the element. (protected)
○ neighbors: List<NetworkElement> - Connected adjacent elements. (protected, initial value = empty list)

● **Methods**

○ void addNeighbor(NetworkElement element) - Adds a neighbor.
○ void removeNeighbor(NetworkElement element) - Removes a neighbor.
○ List<NetworkElement> getNeighbors() - Returns neighbors.
○ boolean isAdjacentTo(NetworkElement element) - Checks adjacency.
○ int getId() - Returns element ID.

8.1.15 Main

● **Responsibility**
Entry point of the program. Handles user input/output, displays menus, collects game setup data, and starts the game loop.

● **Superclasses**
None

● **Interfaces**
None

● **Attributes**

○ scanner: Scanner - Reads input from console. (private static, initial value = new Scanner(System.in))

● **Methods**
○ void main(String[] args) - Displays menu and handles user choices.
○ void startGame() - Collects player data, creates teams, initializes system, and starts game.
○ void runGameLoop(System system) - Executes game loop until end condition.
○ boolean handlePlayerTurn(Player player, System system) - Handles player actions.

○ boolean handlePlumberTurn(Plumber plumber, System system) - Handles plumber actions.
○ boolean handleSaboteurTurn(Saboteur saboteur, System system) - Handles saboteur actions.
○ int readInt(int defaultValue) - Reads integer input or returns default on failure.
○ void showInstructions() - Displays game instructions.

8.2 Detailed plan of testing

8.2.1 Test-case 1

Description

This test case verifies that the system can successfully start a new game with valid input data and correctly initialize the game state. It tests the normal execution path of **UC-01: Start Game**, including player creation, team assignment, network initialization, and transition into the first game tick.

Unit of functionality to be tested

- Main menu handling (Main.main)
- Game setup process (Main.startGame)
- Team creation and player assignment (Team)
- System initialization (System.startGame, System.initializeGame)
- Initial network creation (PipeNetwork)
- Timer activation (GameTimer.start)
- Initial player placement (Player.setPosition)
- Start of game loop (System.updateRoundEvents, WaterFlowManager.recalculateFlow)

Input

Initialization

- No existing game is running
- Program is at main menu
- Tester will enter valid setup data
- System starts from clean state

Control (commands)

➔ 1
➔ 4
➔ a
➔ 1
➔ b
➔ 2
➔ c
➔ 1
➔ d
➔ 2
➔ 1
➔ no
➔ no

- ➔ no
- ➔ 0

Output

```
Welcome to Pipes in the Desert!
--- Game Setup ---
[Team] Team created: Plumbers
[Team] Team created: Saboteurs
a joined the Plumbers.
b joined the Saboteurs.
c joined the Plumbers.
d joined the Saboteurs.
--- Starting Game ---
System.startGame()
System.initializeGame()
[PipeNetwork] Created empty network.
[PipeNetwork] Element added: Spring (id: 1)
[PipeNetwork] Element added: Cistern (id: 2)
[PipeNetwork] Element added: Pump (id: 3)
[PipeNetwork] Element added: Pipe (id: 4)
[PipeNetwork] Element added: Pipe (id: 5)
[PipeNetwork] Connected Spring (id: 1) <-> Pipe (id: 4)
[PipeNetwork] Connected Pipe (id: 4) <-> Pump (id: 3)
[PipeNetwork] Connected Pump (id: 3) <-> Pipe (id: 5)
[PipeNetwork] Connected Pipe (id: 5) <-> Cistern (id: 2)

[Player:a] setPosition() called
[Player:b] setPosition() called
[Player:c] setPosition() called
[Player:d] setPosition() called

GameTimer.start()
[Player:a] startTurn() called

========== GAME TICK ==========
System.updateRoundEvents()
System.requestComponentGeneration()

Generate new pipe at cistern? (yes/no):
Generate new pump at cistern? (yes/no):
Trigger random pump breakdown? (yes/no):

WaterFlowManager.recalculateFlow()
System.getCurrentPlayer()

--- a's turn (Plumber) ---

Turn ended without action.

System.nextTurn()
```

[Player:a] endTurn() called
[Player:c] startTurn() called

8.2.2 Test-case 2

Description

This test case verifies that the system rejects an invalid game setup when the minimum team-size requirement is not satisfied. It tests the failure path of **UC-01: Start Game**, where the Tester provides a valid number of players but assigns fewer than two players to one of the teams. The expected behavior is that the system displays the validation error, returns to the main menu, and does not start the game session.

Unit of functionality to be tested

- Main menu handling (Main.main)
- Game setup validation (Main.startGame)
- Team creation and member assignment (Team)
- Minimum team-size checking before starting the session
- Return-to-main-menu behavior after invalid setup

Input

Initialization

- No existing game is running
- Program is at main menu
- Tester will enter setup data with an invalid team distribution
- System starts from clean state

Control (commands)

➔ 1
➔ 4
➔ a
➔ 1
➔ b
➔ 2
➔ c
➔ 2
➔ d
➔ 2
➔ 1

Output

Welcome to Pipes in the Desert!
--- Game Setup ---
[Team] Team created: Plumbers
[Team] Team created: Saboteurs

How many players? (minimum 4):
Player 1 name:
Choose team for a:
a joined the Plumbers.

Player 2 name:
Choose team for b:
b joined the Saboteurs.

Player 3 name:
Choose team for c:
c joined the Saboteurs.

Player 4 name:
Choose team for d:
d joined the Saboteurs.

Choose game duration:

Each team must have at least 2 players. Plumbers: 1, Saboteurs: 3
Returning to main menu.
Welcome to Pipes in the Desert!

8.2.3 Test-case 3

Description

This test case verifies that one normal game tick is executed correctly during an already running game. It tests the basic behavior of **UC-02: Run Game**, including controlled random-event prompts, automatic water-flow recalculation, display of the active player, and normal turn advancement after the player ends the turn.

Unit of functionality to be tested

- System.updateRoundEvents()
- System.requestComponentGeneration()
- WaterFlowManager.recalculateFlow()
- System.getCurrentPlayer()
- System.nextTurn()
- score and timer display at the end of the tick

Input

Initialization

- A game is already running successfully
- The default initial network exists:
    - Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- All players are on Pump(3)
- Current player is a (Plumber)

- No pump is broken
- No pipe is punctured
- Cistern has no generated spare pipes or pumps

Control (commands)

➔ no
➔ no
➔ no
➔ 0

Output

```
========== GAME TICK ==========
System.updateRoundEvents()
System.requestComponentGeneration()
Generate new pipe at cistern? (yes/no):
Generate new pump at cistern? (yes/no):
Trigger random pump breakdown? (yes/no):
WaterFlowManager.recalculateFlow()
System.getCurrentPlayer()
--- a's turn (Plumber) ---
Turn ended without action.
System.nextTurn()
[Player:a] endTurn() called.
[Player:c] startTurn() called.
[Scores] Plumbers: <int> | Saboteurs: <int>
[Timer] Remaining: <int>
System.checkEndCondition()
```

8.2.4 Test-case 4

Description

This test case verifies the successful movement of a player from the initial pump to a valid unoccupied pipe. It tests the success path of **UC-03: Move Player**, including destination display, movement acceptance, position update, and normal turn completion.

Unit of functionality to be tested

- movement menu handling
- Player.moveTo(NetworkElement target)
- position update
- pipe occupancy update
- turn completion after successful move

Input

Initialization

- A game is already running
- The default initial network exists:

    - Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Pipe(4) is unoccupied
- Pipe(5) is unoccupied
- No pipe is punctured
- No pump is broken

Control (commands)

➔ 1
➔ 4

Output

--- a's turn (Plumber) ---
=== Network Status ===
Current position: Pump (ID: 3)
Actions:
1 Move
...
0 End turn
Available destinations:
Pump (ID: 3)
Pipe (ID: 4)
Pipe (ID: 5)
Enter target element ID:
[Player:a] moveTo() called.
[Player:a] Moved to Pipe.
System.nextTurn()
[Player:a] endTurn() called.

8.2.5 Test-case 5

Description

This test case verifies that an invalid movement attempt is rejected and does not consume the current player's turn. It tests the rejection path of **UC-03: Move Player**, specifically when the Tester enters the ID of a non-movable element.

Unit of functionality to be tested

- invalid destination handling in movement
- rejection behavior in Player.moveTo(...)
- turn preservation after failed movement
- same-player retry inside the same tick

Input

Initialization

- A game is already running

- The default initial network exists:
  - Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- No pipe is punctured
- No pump is broken
- Spring(1) exists in the network and is not a valid movement destination

Control (commands)

➔ 1
➔ 1
➔ 0

Output

--- a's turn (Plumber) ---
=== Network Status ===
Current position: Pump (ID: 3)
Actions:
1 Move
...
0 End turn
Available destinations:
Pump (ID: 3)
Pipe (ID: 4)
Pipe (ID: 5)
Enter target element ID:
[Player:a] moveTo() called.
[Player:a] moveTo() rejected
Invalid move, try another!
Invalid or rejected action. Please choose again.

--- a's turn (Plumber) ---
=== Network Status ===
Current position: Pump (ID: 3)
Actions:
...
0 End turn
Turn ended without action.
System.nextTurn()
[Player:a] endTurn() called.

8.2.6 Test-case 6

Description

This test case verifies that a Saboteur standing on an intact pipe can successfully puncture it. It tests the successful path of **UC-04: Saboteur Punctures Pipe**, including action selection, confirmation, pipe-state change, and flow recalculation.

Unit of functionality to be tested

- saboteur action handling
- Saboteur.puncturePipe(Pipe pipe)
- Pipe.puncture()
- confirmation prompt handling
- flow recalculation after puncture

Input

Initialization

- A game is already running
- The default initial network exists:
  - Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is b (Saboteur)
- Player b is standing on Pipe(4)
- Pipe(4) is intact
- No pump is broken
- No other pipe is punctured

Control (commands)

➔ 2
➔ yes

Output

--- b's turn (Saboteur) ---
=== Network Status ===
Pipe (ID:4) [OK | Occupied by: b]
Current position: Pipe (ID: 4)
Actions:
1 Move
2 Puncture pipe
3 Change pump direction
0 End turn
Puncture the pipe? (yes/no):
[Saboteur:b] puncturePipe() called.
[Pipe:4] puncture() called.
[Pipe:4] Pipe is now punctured and leaking.
WaterFlowManager.recalculateFlow()
System.nextTurn()
[Player:b] endTurn() called.

8.2.7 Test-case 7

Description

This test case verifies that a Plumber can successfully repair a punctured pipe. It tests the successful execution path of **UC-05: Repair Pipe**, including position validation, confirmation prompt handling, pipe state restoration, and water-flow recalculation after the action.

Unit of functionality to be tested

- plumber action handling
- Plumber.repairPipe(Pipe pipe)
- Pipe.repair()
- confirmation prompt handling
- validation that player is standing on a pipe
- water-flow recalculation after repair
- turn completion after successful repair

Input

Initialization

- A game is already running
- The default initial network exists:
  - Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pipe(4)
- Pipe(4) is **punctured**
- No pump is broken
- No other pipe is punctured

Control (commands)

➔ 2
➔ yes

Output

--- a's turn (Plumber) ---
=== Network Status ===
Pipe (ID:4) [PUNCTURED | Occupied by: a]
Current position: Pipe (ID: 4)

Actions:
1 Move
2 Repair pipe
3 Repair pump
...
0 End turn
Repair the pipe? (yes/no):
[Plumber:a] repairPipe() called.
[Pipe:4] repair() called.
[Pipe:4] Pipe repaired successfully.
WaterFlowManager.recalculateFlow()
System.nextTurn()
[Player:a] endTurn() called.

8.2.8 Test-case 8

Description

This test case verifies that a Plumber can successfully change the direction of a pump. It tests the successful execution path of **UC-06: Change Pump Direction**, including selection of input/output pipes, confirmation, update of the pump's direction, and subsequent water-flow recalculation.

Unit of functionality to be tested

- plumber action handling for pump interaction
- Pump.setDirection(Pipe in, Pipe out) (or equivalent)
- validation that the player is on a pump
- selection of input/output pipes
- confirmation prompt handling
- water-flow recalculation after direction change
- turn completion after successful change

Input

Initialization

- A game is already running
- The default initial network exists:
  - Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Pump(3) is not broken
- Pump(3) is currently set to transfer water from Pipe(4) to Pipe(5)
- Both Pipe(4) and Pipe(5) are connected to Pump(3)

Control (commands)

➔ 4
➔ 5
➔ 4
➔ yes

Output

--- a's turn (Plumber) ---
=== Network Status ===
Pump (ID:3) [OK]
Current position: Pump (ID: 3)

Actions:
1 Move
2 Repair pipe
3 Repair pump
4 Change pump direction
...
0 End turn

Select input pipe ID:
Select output pipe ID:

Confirm direction change? (yes/no):

[Pump:3] setDirection() called.
[Pump:3] Direction updated successfully.
WaterFlowManager.recalculateFlow()

System.nextTurn()
[Player:a] endTurn() called.

8.2.9 Test-case 9

Description

This test case verifies that a Plumber can successfully connect a free pipe end to an adjacent network element. It tests the successful execution path of **UC-07: Connect Pipe End**, including selecting a valid pipe, selecting a valid adjacent element, confirmation of the action, updating the network connections, and recalculation of water flow.

Unit of functionality to be tested

- plumber action handling for pipe connection
- Pipe.connectFreeEnd(NetworkElement element) (or equivalent)
- validation of adjacency between pipe and target element
- validation that the pipe has a free end
- confirmation prompt handling
- update of neighbor relationships in the network
- water-flow recalculation after connection
- turn completion after successful connection

Input

Initialization

- A game is already running
- The network contains:
    - Pump(3) connected to Pipe(4)
- Pipe(4) has one **free end**
- There exists an adjacent element (e.g., Cistern(2)) that is not yet connected to Pipe(4)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Pump and pipe are not broken or punctured

Control (commands)

➔ 5
➔ 4
➔ 2
➔ yes

Output

--- a's turn (Plumber) ---

=== Network Status ===
Pipe (ID:4) [FREE END]
Current position: Pump (ID: 3)

Actions:
...
5 Connect pipe end
...

Select pipe ID:
Select target element ID:
Confirm connection? (yes/no):

[Pipe:4] connectFreeEnd() called.
[Pipe:4] Connected to element 2.
WaterFlowManager.recalculateFlow()

System.nextTurn()
[Player:a] endTurn() called.

8.2.10 Test-case 10

Description This test case verifies that a Plumber standing on a broken pump can successfully repair it. It tests the successful execution path of **UC-06: Plumber Repairs Broken Pump**, including position validation, confirmation prompt handling, pump state restoration, and water-flow recalculation.

Unit of functionality to be tested

- Plumber.repairPump(Pump pump)
- Pump.repair()
- confirmation prompt handling
- validation that player is standing on a pump
- water-flow recalculation after repair
- turn completion after successful repair

Possible failures

- valid repair is rejected
- pump state does not change from broken to working
- flow is not recalculated after repair
- action succeeds when player is not standing on a pump
- turn does not end after successful repair

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)

- Player a is standing on Pump(3)
- Pump(3) is BROKEN (triggered in a previous tick via breakdown prompt)

Control (commands):

- 3
- yes

Output

```
--- a's turn (Plumber) ---
=== Network Status ===
Pump (ID:3) [BROKEN]
Current position: Pump (ID: 3)
Actions:
1 Move
2 Repair pipe
3 Repair pump
...
0 End turn
Repair the pump? (yes/no):
[Plumber:a] repairPump() called. Pump broken: true
[Pump:3] repair() called.
[Pump:3] Pump repaired and restored to working state.
[Plumber:a] Pump repaired. Water transfer can resume.
WaterFlowManager.recalculateFlow()
System.nextTurn()
[Player:a] endTurn() called.
```

8.2.11 Test-case 11

Description This test case verifies that a Saboteur standing on a pump can successfully change its direction. It tests the successful execution path of **UC-08: Saboteur Changes Pump Direction**, including connected pipe display, index selection, direction update, and water-flow recalculation.

Unit of functionality to be tested

- saboteur action handling for pump interaction
- Saboteur.changePumpDirection(Pump, Pipe, Pipe)
- Pump.setDirection(Pipe input, Pipe output)
- validation that player is standing on a pump
- connected pipe index display and selection
- water-flow recalculation after direction change
- turn completion after successful change

Possible failures

- valid direction change is rejected
- pump direction is not updated
- wrong pipes are assigned as input/output

- action succeeds when player is not on a pump
- flow is not recalculated after direction change
- turn does not end after successful action

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is b (Saboteur)
- Player b is standing on Pump(3)
- Pump(3) is working
- Pump(3) current direction: input=Pipe(4), output=Pipe(5)

Control (commands):

- 3
- 1
- 0

Output

```
--- b's turn (Saboteur) ---
=== Network Status ===
Pump (ID:3) [OK]
Current position: Pump (ID: 3)
Actions:
1 Move
2 Puncture pipe
3 Change pump direction
0 End turn
Connected pipes:
  0: Pipe 4
  1: Pipe 5
Select input pipe index:
Select output pipe index:
[Saboteur:b] changePumpDirection() called.
[Pump:3] setDirection() called.
[Pump:3] Direction updated successfully.
[Saboteur:b] Pump direction changed successfully. Water flow may now be diverted away from cisterns.
WaterFlowManager.recalculateFlow()
System.nextTurn()
[Player:b] endTurn() called.
```

8.2.12 Test-case 12

Description This test case verifies that a Plumber can successfully disconnect one end of an unoccupied pipe. It tests the successful execution path of **UC-09: Plumber Disconnects a**

Pipe, including pipe selection, disconnection, free-end state update, and water-flow recalculation.

Unit of functionality to be tested

- Plumber action handling for pipe disconnection
- Pipe.disconnectEnd()
- validation that the pipe is not occupied
- free-end state update after disconnection
- water-flow recalculation after disconnection
- turn completion after successful disconnection

Possible failures

- valid disconnection is rejected
- pipe free-end state is not updated
- flow is not recalculated after disconnection
- action succeeds on an occupied pipe
- turn does not end after successful action

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Pipe(5) is unoccupied

Control (commands):

- 6
- 5

Output

```
--- a's turn (Plumber) ---
=== Network Status ===
Pipe (ID:5) [OK]
Current position: Pump (ID: 3)
Actions:
...
6 Disconnect pipe end
...
0 End turn
Enter pipe ID to disconnect:
[Pipe:5] disconnectEnd() called.
[Pipe:5] One end of the pipe is now free.
Pipe end disconnected.
WaterFlowManager.recalculateFlow()
System.nextTurn()
```

[Player:a] endTurn() called.

8.2.13 Test-case 13

Description This test case verifies that disconnecting a pipe occupied by a player is rejected. It tests the failure path of **UC-09: Plumber Disconnects a Pipe**, where the target pipe is currently occupied by another player.

Unit of functionality to be tested

- Pipe.isOccupied() check before disconnection
- rejection behavior when pipe is occupied
- correct error message display
- turn preservation after failed action

Possible failures

- occupied pipe is disconnected
- error message is missing or incorrect
- turn ends after rejected action

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Player c (Plumber) is standing on Pipe(4) - Pipe(4) is occupied

Control (commands):

- 6
- 4
- 0

Output

```
--- a's turn (Plumber) ---
=== Network Status ===
Pipe (ID:4) [OK | Occupied by: c]
Current position: Pump (ID: 3)
Enter pipe ID to disconnect:
Pipe is occupied, cannot disconnect.
Invalid or rejected action. Please choose again.
--- a's turn (Plumber) ---
...
Turn ended without action.
System.nextTurn()
[Player:a] endTurn() called.
```

8.2.14 Test-case 14

Description This test case verifies that a Plumber can successfully pick up a generated pump from the first cistern. It tests the successful execution path of **UC-11: Plumber Picks Up Pump at Cistern**, including availability check, confirmation prompt, inventory update, and cistern count decrease.

Unit of functionality to be tested

- Plumber.collectPumpFromCistern(Cistern)
- Cistern.hasAvailablePump() and Cistern.takePump()
- confirmation prompt handling
- carriedPump state update
- turn completion after successful pickup

Possible failures

- pickup is rejected when pump is available
- carriedPump is not set to true
- cistern available pump count is not decremented
- turn does not end after successful pickup

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Cistern(2) has 1 available pump (generated via yes to pump generation prompt in a previous tick)
- Player a is not already carrying a pump

Control (commands):

- 7
- yes

Output

```
--- a's turn (Plumber) ---
=== Network Status ===
Cistern (ID:2) [Pipes:0 Pumps:1]
Current position: Pump (ID: 3)
Actions:
...
7 Pick up pump from cistern
...
0 End turn
Pick up pump from cistern? (yes/no):
[Plumber:a] collectPumpFromCistern() called.
```

[Plumber:a] Pump collected from cistern. carriedPump = true.
System.nextTurn()
[Player:a] endTurn() called.

8.2.15 Test-case 15

Description This test case verifies that a Plumber can successfully pick up a generated pipe from the first cistern. It tests the successful execution path of **UC-12: Plumber Picks Up Pipe from Cistern**, including availability check, confirmation prompt, inventory update, and cistern count decrease.

Unit of functionality to be tested

- Plumber.collectPipeFromCistern(Cistern)
- Cistern.hasAvailablePipe() and Cistern.takePipe()
- confirmation prompt handling
- carriedPipe state update
- turn completion after successful pickup

Possible failures

- pickup is rejected when pipe is available
- carriedPipe is not set to true
- cistern available pipe count is not decremented
- turn does not end after successful pickup

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Cistern(2) has 1 available pipe (generated via yes to pipe generation prompt in a previous tick)
- Player a is not already carrying a pipe

Control (commands):

- 8
- yes

Output

--- a's turn (Plumber) ---
=== Network Status ===
Cistern (ID:2) [Pipes:1 Pumps:0]
Current position: Pump (ID: 3)
Actions:
...
8 Pick up pipe from cistern

...
0 End turn
Pick up pipe from cistern? (yes/no):
[Plumber:a] collectPipeFromCistern() called.
[Plumber:a] Pipe collected from cistern. carriedPipe = true.
System.nextTurn()
[Player:a] endTurn() called.

8.2.16 Test-case 16

Description This test case verifies that a Plumber carrying a pipe can place it into the network between two existing elements. It tests the successful execution path of **UC-13: Plumber Places New Pipe**, including element selection, network update, and water-flow recalculation.

Unit of functionality to be tested

- Plumber.placeNewPipe(Pipe, NetworkElement, NetworkElement, PipeNetwork)
- PipeNetwork.addElement() and PipeNetwork.connectElements()
- validation that player is carrying a pipe
- new pipe ID generation (network.getPipes().size() + 100)
- network structure update
- water-flow recalculation after placement

Possible failures

- action is rejected when player carries a pipe
- new pipe is not added to the network
- connections are not created correctly
- flow is not recalculated after placement
- turn does not end after successful placement

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Player a is carrying a pipe (carriedPipe = true, collected in a previous turn)

Control (commands):

- 10
- 3
- 2

Output

--- a's turn (Plumber) ---
Current position: Pump (ID: 3)
Actions:

...
10 Place new pipe
...
Enter first connection element ID:
Enter second connection element ID:
[Plumber:a] placeNewPipe() called. Connecting between: Pump and Cistern
[PipeNetwork] Element added: Pipe (id: 102).
[PipeNetwork] Connected Pipe (id: 102) <-> Pump (id: 3).
[PipeNetwork] Connected Pipe (id: 102) <-> Cistern (id: 2).
[Plumber:a] New pipe placed into network and connected. carriedPipe = false.
WaterFlowManager.recalculateFlow()
System.nextTurn()
[Player:a] endTurn() called..

8.2.17 Test-case 17

Description This test case verifies that a Plumber carrying a pump can insert it into an existing pipe by splitting that pipe into two segments. It tests the successful execution path of **UC-14: Plumber Inserts New Pump**, including pipe splitting, segment creation, network reconnection, and water-flow recalculation.

Unit of functionality to be tested

- Plumber.insertPump(Pipe, Pump, PipeNetwork)
- PipeNetwork.insertPump(Pipe, Pump)
- validation that player is carrying a pump
- new pump ID generation (pipeId * 10 + 1)
- pipe split and segment creation (pipeId * 100 + 1, pipeId * 100 + 2)
- original neighbor reconnection
- water-flow recalculation after insertion

Possible failures

- action is rejected when player carries a pump
- original pipe is not removed
- segments are not correctly created and connected
- pump is not inserted between segments
- flow is not recalculated after insertion
- turn does not end after successful insertion

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Player a is standing on Pump(3)
- Player a is carrying a pump (carriedPump = true, collected in a previous turn)

Control (commands):

- 9
- 5

Output

--- a's turn (Plumber) ---
Current position: Pump (ID: 3)
Actions:
...
9 Insert pump into pipe
...
Enter target pipe ID to split:
[Pump:51] Created. connectionLimit: 4
[Plumber:a] insertPump() called.
[PipeNetwork] insertPump() called.
[Pipe:501] Created. capacity: 10
[Pipe:502] Created. capacity: 10
[PipeNetwork] Element added: Pump (id: 51).
[PipeNetwork] Element added: Pipe (id: 501).
[PipeNetwork] Element added: Pipe (id: 502).
[PipeNetwork] Connected Pipe (id: 501) <-> Pump (id: 51).
[PipeNetwork] Connected Pump (id: 51) <-> Pipe (id: 502).
[PipeNetwork] Connected Pump (id: 3) <-> Pipe (id: 501).
[PipeNetwork] Connected Cistern (id: 2) <-> Pipe (id: 502).
[PipeNetwork] Pump inserted successfully. Original pipe (id: 5) split into segments 501 and 502.
[Plumber:a] Pump inserted into pipe. Network updated. carriedPump = false.
WaterFlowManager.recalculateFlow()
System.nextTurn()
[Player:a] endTurn() called.

8.2.18 Test-case 18

Description This test case verifies that the system can deterministically trigger a pump breakdown during a game tick. It tests **UC-15: System Randomly Breaks Pump**, replacing random behavior with a tester-controlled yes/no prompt.

Unit of functionality to be tested

- System.updateRoundEvents()
- PipeNetwork.getWorkingPumps()
- Pump.breakDown()
- WaterFlowManager.recalculateFlow() after breakdown

Possible failures

- breakdown prompt does not appear
- pump is not broken after yes response
- working pump is not selected correctly
- flow is not recalculated after breakdown
- breakdown prompt triggers when no working pumps exist

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Pump(3) is working

Control (commands):

- no
- no
- yes
- 0

Output

```
========== GAME TICK ==========
System.updateRoundEvents()
System.requestComponentGeneration()
Generate new pipe at cistern? (yes/no):
Generate new pump at cistern? (yes/no):
Trigger random pump breakdown? (yes/no):
[Pump:3] breakDown() called.
[Pump:3] Pump is now broken.
WaterFlowManager.recalculateFlow()
System.getCurrentPlayer()
--- a's turn (Plumber) ---
=== Network Status ===
Pump (ID:3) [BROKEN]
Current position: Pump (ID: 3)
...
Turn ended without action.
System.nextTurn()
[Player:a] endTurn() called.
```

8.2.19 Test-case 19

Description This test case verifies that the system can generate both a new pipe and a new pump at a cistern during a single game tick. It tests **UC-16: System Generates New Pipe** and **UC-17: System Generates New Pump** together.

Unit of functionality to be tested

- System.requestComponentGeneration()
- Cistern.generatePipe() and Cistern.generatePump()
- cistern available count updates
- correct display of updated cistern state in network status

Possible failures

- generation prompts do not appear
- pipe or pump count does not increment
- cistern display does not reflect updated counts
- generation occurs when prompt answer is no

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber)
- Cistern(2) has 0 available pipes, 0 available pumps

Control (commands):

- yes
- yes
- no
- 0

Output

```
========== GAME TICK ==========
System.updateRoundEvents()
System.requestComponentGeneration()
Generate new pipe at cistern? (yes/no):
[Cistern:2] generatePipe() called.
[Cistern:2] New pipe generated. Available pipes: 1
Generate new pump at cistern? (yes/no):
[Cistern:2] generatePump() called.
[Cistern:2] New pump generated. Available pumps: 1
Trigger random pump breakdown? (yes/no):
WaterFlowManager.recalculateFlow()
System.getCurrentPlayer()
--- a's turn (Plumber) ---
=== Network Status ===
Cistern (ID:2) [Pipes:1 Pumps:1]
Current position: Pump (ID: 3)
...
Turn ended without action.
System.nextTurn()
[Player:a] endTurn() called.
```

8.2.20 Test-case 20

Description This test case verifies that water is successfully delivered to a cistern when the network path from spring to cistern is fully intact. It tests the plumber-scoring path of **UC-18: Water Leakage and Flow Logic**.

Unit of functionality to be tested

- WaterFlowManager.recalculateFlow()
- WaterFlowManager.processSpring(Spring)
- Cistern.receiveWater(int)
- WaterFlowManager.registerDeliveredWater(int)
- ScoreBoard.addDeliveredWater(int)
- plumber score increase

Possible failures

- water is not delivered despite intact path
- delivered water amount is incorrect
- plumber score is not updated
- saboteur score incorrectly increases

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- All pipes intact, pump working, pump direction: input=Pipe(4), output=Pipe(5)
- Current player is a (Plumber)

Control (commands):

- no
- no
- no
- 0

Output

```
========== GAME TICK ==========
System.updateRoundEvents()
System.requestComponentGeneration()
Generate new pipe at cistern? (yes/no):
Generate new pump at cistern? (yes/no):
Trigger random pump breakdown? (yes/no):
WaterFlowManager.recalculateFlow()
WaterFlowManager.processSpring(spring)
[Cistern:2] receiveWater() called. Amount: 10
[Cistern:2] Water received successfully. Total stored water: 10
WaterFlowManager.registerDeliveredWater(10)
ScoreBoard.addDeliveredWater(10)
...
Turn ended without action.
System.nextTurn()
[Scores] Plumbers: 10 | Saboteurs: 0
[Timer] Remaining: <int>
System.checkEndCondition()
```

8.2.21 Test-case 21

Description

This test case verifies that water leaks when it reaches a punctured pipe. It tests the saboteur-scoring path of UC-18: Water Leakage and Flow Logic via a punctured pipe.

Unit of functionality to be tested

· WaterFlowManager.processSpring(Spring)
· Pipe.isPunctured() check during flow
· WaterFlowManager.registerLeakedWater(int)
· ScoreBoard.addLeakedWater(int)
· saboteur score increase

Possible failures

· water is delivered despite punctured pipe
· saboteur score is not updated
· plumber score incorrectly increases
· leaked water amount is incorrect

Input

Initialization:
· A game is already running
· Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
· Pipe(4) is PUNCTURED
· Current player is a (Plumber)

Control (commands):

· no
· no
· no
· 0

Output

```
========== GAME TICK ==========
System.updateRoundEvents()
System.requestComponentGeneration()
Generate new pipe at cistern? (yes/no):
Generate new pump at cistern? (yes/no):
Trigger random pump breakdown? (yes/no):
WaterFlowManager.recalculateFlow()
WaterFlowManager.processSpring(spring)
WaterFlowManager.registerLeakedWater(10)
ScoreBoard.addLeakedWater(10)

Turn ended without action.
System.nextTurn()
[Scores] Plumbers: 0 | Saboteurs: 10
```

8.2.22 Test-case 22

Description
This test case verifies that the system correctly ends the game when the timer expires and declares the winner based on the scores.

Unit of functionality to be tested
· System.checkEndCondition()
· GameTimer.hasExpired()
· System.endGame()
· ScoreBoard.determineWinner()

Possible failures
· game does not end when timer expires
· winner is not declared
· incorrect winner is selected
· timer expiration is ignored

Input
Initialization:
· A game is already running
· Timer remaining time = 1 tick
· Scores: Plumbers = 20, Saboteurs = 10

Control (commands):
· no
· no
· no
· 0

Output
========== GAME TICK ==========
System.updateRoundEvents()
System.requestComponentGeneration()
Generate new pipe at cistern? (yes/no):
Generate new pump at cistern? (yes/no):
Trigger random pump breakdown? (yes/no):
WaterFlowManager.recalculateFlow()

System.checkEndCondition()
GameTimer.hasExpired()

System.endGame()
Winner: Plumbers

8.2.23 Test-case 23

Description This test case verifies that water leaks when it reaches a pipe with a free end. It tests the saboteur-scoring path of **UC-18: Water Leakage and Flow Logic** via a disconnected pipe.

Unit of functionality to be tested

- WaterFlowManager.processSpring(Spring)
- Pipe.hasFreeEnd() check during flow
- WaterFlowManager.registerLeakedWater(int)
- ScoreBoard.addLeakedWater(int)

Possible failures

- water is delivered despite pipe having free end
- saboteur score is not updated
- plumber score incorrectly increases

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Pipe(4) has a FREE END (disconnected in a previous turn via action 6)
- No pipe is punctured, pump is working

Control (commands):

- no
- no
- no
- 0

Output

```
========== GAME TICK ==========
System.updateRoundEvents()
...
WaterFlowManager.recalculateFlow()
WaterFlowManager.processSpring(spring)
WaterFlowManager.registerLeakedWater(10)
ScoreBoard.addLeakedWater(10)
...
Turn ended without action.
System.nextTurn()
[Scores] Plumbers: 0 | Saboteurs: 10
[Timer] Remaining: <int>
System.checkEndCondition()
```

8.2.24 Test-case 24

Description This test case verifies that water leaks when the pump routes it to a pipe that does not lead to a cistern. It tests the saboteur-scoring path of **UC-18: Water Leakage and Flow Logic** via misdirected pump routing.

Unit of functionality to be tested

- WaterFlowManager.processSpring(Spring)

- Pump.getActiveOutput() routing to a pipe with a free end on the cistern side
- WaterFlowManager.registerLeakedWater(int)
- ScoreBoard.addLeakedWater(int)

Possible failures

- water is delivered despite wrong routing
- saboteur score is not updated
- plumber score incorrectly increases

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Pipe(5) has a FREE END - its connection to Cistern(2) was disconnected in a previous turn
- Pump direction unchanged: input=Pipe(4), output=Pipe(5)
- Pipe(4) is intact

Control (commands):

- no
- no
- no
- 0

Output

```
========== GAME TICK ==========
System.updateRoundEvents()
...
WaterFlowManager.recalculateFlow()
WaterFlowManager.processSpring(spring)
WaterFlowManager.registerLeakedWater(10)
ScoreBoard.addLeakedWater(10)
...
Turn ended without action.
System.nextTurn()
[Scores] Plumbers: 0 | Saboteurs: 10
[Timer] Remaining: <int>
System.checkEndCondition()
```

8.2.25 Test-case 25

Description This test case verifies that each player is limited to exactly one action per turn. It tests the one-action-per-turn constraint defined in FR-20. After a valid action completes, the turn loop exits immediately and control passes to the next player without giving the same player a second action.

Unit of functionality to be tested

- Main.runGameLoop() while-loop exit condition after valid action
- Player.moveTo() returning true
- System.nextTurn() called immediately after first valid action
- turn ownership correctly transferred to next player

Possible failures

- same player is prompted for a second action after completing a move
- turn does not end after first valid action
- next player is not activated after turn ends

Input

Initialization:

- A game is already running
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- Current player is a (Plumber), standing on Pump(3)
- Pipe(4) is unoccupied

Control (commands):

- 1
- 4

Output

```
--- a's turn (Plumber) ---
=== Network Status ===
Current position: Pump (ID: 3)
Actions:
1 Move
...
Available destinations:
Pipe (ID: 4)
Pipe (ID: 5)
Pump (ID: 3)
Enter target element ID:
[Player:a] moveTo() called.
[Player:a] Moved to Pipe.
System.nextTurn()
[Player:a] endTurn() called.
[Player:b] startTurn() called.
--- b's turn (Saboteur) ---
```

8.2.26 Test-case 26

Description This test case verifies that the game correctly ends with the Saboteur team winning when the timer expires. It tests **UC-19: End Game and Declare Winner** in the saboteur-win scenario. A pipe is punctured early in the game to ensure water leaks throughout, giving saboteurs a higher score.

Unit of functionality to be tested

- System.checkEndCondition() and GameTimer.hasExpired()
- System.endGame()
- ScoreBoard.determineWinner()
- correct winner announcement
- game session termination

Possible failures

- game does not end when timer expires
- wrong winner is declared
- game over message is missing or incorrect
- plumber score incorrectly exceeds saboteur score

Input

Initialization:

- A new game is started with 4 players: a (Plumber), b (Saboteur), c (Plumber), d (Saboteur)
- Duration: option 1 (3 ticks)
- Default network generated on start

Control (commands) - abbreviated pattern:

Round 1:

- no / no / no (tick events for a's turn)
- 0 (player a ends turn)
- no / no / no (tick events for b's turn)
- 2 / yes (b punctures Pipe(4))
- no / no / no (tick events for c's turn)
- 0 (player c ends turn)
- no / no / no (tick events for d's turn)
- 0 (player d ends turn - timer ticks to 2)

Rounds 2 and 3: each player enters 0 each turn (all end turn without action), no events triggered (no / no / no before each turn).

After Round 3: timer ticks to 0 → game ends.

Output

[Saboteur:b] puncturePipe() called.
[Pipe:4] puncture() called.
[Pipe:4] Pipe is now punctured and leaking.
WaterFlowManager.recalculateFlow()
WaterFlowManager.registerLeakedWater(10)
ScoreBoard.addLeakedWater(10)
...
[after round 3, timer hits 0]

...
[Scores] Plumbers: 0 | Saboteurs: <int>
[Timer] Remaining: 0
System.checkEndCondition()
GameTimer.hasExpired()

========== GAME OVER ==========

System.endGame()
GameTimer.stop()
ScoreBoard.determineWinner()
Winner: Saboteurs

8.2.27 Test-case 27

Description This test case verifies that the game correctly ends with the Plumber team winning when the timer expires. It tests **UC-19: End Game and Declare Winner** in the plumber-win scenario. The network remains intact throughout all rounds so water is delivered every tick.

Unit of functionality to be tested

- System.checkEndCondition() and GameTimer.hasExpired()
- System.endGame()
- ScoreBoard.determineWinner()
- correct winner announcement when plumberScore > saboteurScore

Possible failures

- game does not end when timer expires
- wrong winner is declared
- game over message is missing or incorrect
- saboteur score incorrectly exceeds plumber score

Input

Initialization:

- A new game is started with 4 players: a (Plumber), b (Saboteur), c (Plumber), d (Saboteur)
- Duration: option 1 (3 ticks)
- Default network: Spring(1) → Pipe(4) → Pump(3) → Pipe(5) → Cistern(2)
- All pipes intact, pump working throughout all rounds

Control (commands):

- All 3 rounds: each player enters 0 (end turn), all tick events answered no / no / no

After Round 3: timer ticks to 0 → game ends.

Output

WaterFlowManager.recalculateFlow()
WaterFlowManager.processSpring(spring)
[Cistern:2] receiveWater() called. Amount: 10
WaterFlowManager.registerDeliveredWater(10)
ScoreBoard.addDeliveredWater(10)
...
[after round 3, timer hits 0]
...
[Scores] Plumbers: <int> | Saboteurs: 0
[Timer] Remaining: 0
System.checkEndCondition()
GameTimer.hasExpired()

========== GAME OVER ==========

System.endGame()
GameTimer.stop()
ScoreBoard.determineWinner()
Winner: Plumbers

8.3 Plans of the supporting programs

The following supporting programs are used to execute and verify the test cases defined in Section 8.2.

8.3.1 Test Input Loader

Description
Loads predefined test command sequences from a file and feeds them to the program instead of manual console input.

Functionality
· Reads commands line-by-line from a file using load <filepath>
· Sends each command to the system as input
· Stops when end-of-file (EOF) is reached

Purpose
Ensures repeatable and consistent execution of test cases without manual input.

8.3.2 Output Redirector

Description
Redirects all program output to a file for later comparison.

Functionality
· Activated using save <filepath>
· Writes all output data-streams into the specified file
· Stores file in the current working directory

Purpose
Allows capturing program output for validation against expected results.

8.3.3 Output Comparator

Description
Compares actual program output with expected output defined in test cases.

Functionality
· Reads expected output (from test case definition file)
· Reads actual output (from saved output file)
· Compares line-by-line
· Reports mismatch if any difference is found

Purpose
Verifies correctness of program behavior automatically.

8.3.4 Test Execution Process

Description
Defines how a complete test case is executed using the supporting programs.

Steps
· Prepare input file containing commands (based on Section 8.2)
· Run program using load <input-file>
· Redirect output using save <output-file>
· Execute test until completion
· Use Output Comparator to compare actual and expected output

Purpose
Ensures each test case is executed in a controlled and repeatable manner.

8.4 Tools Used

Tool	Purpose	Description
ChatGPT (AI)	Writing assistance	Used for proofreading, grammar correction, brainstorming ideas, and clarifying edge cases in the documentation.
Microsoft Word	Documentation	Used to write, format, and organize the Detailed Prototype Plans document.
Discord	Communication	Used for online team meetings and coordination between team members.
Google Meet	Communication	Used for real-time collaborative coordination during the working session on April 18th.
GitHub	Version control	Used to manage, store, and share source code and documentation across the team.
Microsoft Teams	Communication	Used for file sharing and informal coordination between team members.

8.5 Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
2026.04.18, 11:00	6 hours	Hameez, Aasif	**Meeting (BME Building Q).** The performers met in person to design the design-level plan of classes. **Decisions:** (1) Defined all private attributes and methods for the core logic classes (System, Player, NetworkElement, etc.) to ensure exact implementation alignment. (2) Cross-checked method signatures against the existing skeleton to maintain consistency. (3) Structured Section 8.1 with detailed algorithmic responsibilities for each class.
2026.04.19, 09:00	6 hours	Ibrahim	**Individual work session.** Ibrahim reviewed the professor's feedback for Task 7 and authored Section 8.0 (Corrections). **Decisions:** (1) Added load and save commands to the input language to allow switching between file-based and console I/O. (2) Specified that output files are saved to the current working directory. (3) Established a global rule for wrong input handling (non-numeric or out-of-range) to be applied across all test scenarios.
2026.04.20, 12:00	6 hours	Arda	**Individual work session.** Arda authored the first nine detailed test cases (Test-case 1 to 9). **Decisions:** (1) Mapped "Start Game," "Move Player," and basic "Repair/Puncture" actions into the required input/output language format. (2) Included expected console output strings

			for each valid step and verified them against the interface definition.
2026.04.20, 18:00	8 hours	Ibrahim, Hameez	**Meeting (Google Meet).** Collaborative session to complete the remaining 18 test cases and supporting program documentation. **Decisions:** (1) Authored Test-cases 10 through 27, covering complex scenarios like pump insertion, water flow logic, and the timer-based end-game. (2) Authored Section 8.3 (Plans of supporting programs). (3) Integrated "wrong input" paths into the scenarios as requested by the supervisor.
2026.04.20, 23:30	1.5 hours	Ibrahim, Hameez	**Meeting (Google Meet).** Final review and assembly of the submission package. **Decisions:** (1) Updated the "Tools Used" table. (2) Performed a final consistency check between the class attributes in Section 8.1 and the test cases in Section 8.2. (3) Fixed document formatting, applied final corrections to the protocol table, and prepared the ZIP for submission.